Virtual STM32 Smart Environmental Monitoring Station Design and Implementation

Development Research Paper

May 10, 2026

Abstract

With the rapid development of Internet of Things (IoT) technology and the widespread application of embedded systems in various industries, intelligent environmental monitoring has become an important component of smart city construction. However, in the process of embedded system teaching and R&D, developers generally face many challenges such as high hardware costs, complex debugging processes, high barriers to understanding sensors, and inconvenient data visualization. Traditional embedded development models require learners to purchase a large amount of hardware equipment and perform repeated debugging on actual circuit boards, which not only increases learning costs but also limits the flexibility and scalability of teaching scenarios.

To address the above problems, this paper proposes and implements a virtualized intelligent environmental monitoring station based on STM32. The core contributions of this research include three aspects: First, a high-fidelity virtual STM32F103C8T6 hardware platform is designed and implemented, fully simulating key peripherals such as the MCU core, memory system, GPIO, ADC, I2C, UART, and timers, providing a software environment in which embedded programs can run without real hardware; Second, based on the virtual hardware platform, virtual drivers for six common environmental sensors are developed, including the DHT11 temperature and humidity sensor, MQ-135 air quality sensor, BH1750 light sensor, BMP280 barometric pressure sensor, and noise sensor, and a unified sensor modeling framework and noise model are established to ensure the authenticity and reliability of simulated data; Finally, a Web visualization system based on Python Flask is built, realizing real-time acquisition, storage, alarm, and chart display of sensor data, interacting with the front end through a RESTful API, and supporting remote adjustment of environmental parameters.

Experimental results show that the instruction execution efficiency of the virtual STM32 platform is highly consistent with real hardware, the sensor simulation accuracy reaches or approaches the nominal values in real sensor datasheets, and the Web system API response time is less than 100 milliseconds, meeting the needs of real-time monitoring. This research provides a low-threshold, high-flexibility experimental platform for embedded system teaching, and also provides an efficient software development environment for IoT application prototype verification, having important theoretical research value and practical application significance.

Keywords: STM32; virtualization; environmental monitoring; sensor simulation; Internet of Things; embedded systems education

Contents

1 Introduction

1.1 Research Background and Significance

The Internet of Things (IoT), as an important component of the new generation of information technology, is profoundly changing the production and lifestyle of human society. In the fields of smart city construction, industrial automation, precision agriculture management, and smart homes, real-time monitoring and intelligent analysis of environmental parameters have become indispensable basic capabilities. Environmental indicators such as temperature, humidity, air quality, light intensity, atmospheric pressure, and noise level directly affect people's health, comfort, and production efficiency. Therefore, building efficient, reliable, and low-cost intelligent environmental monitoring systems has important social value and economic significance.

Embedded systems, as the core technology of IoT terminal devices, undertake the key tasks of sensor data acquisition, processing, and transmission. The STM32 series microcontrollers have become the mainstream choice in the embedded development field by virtue of their high performance, low power consumption, rich peripheral interfaces, and complete ecosystem support. Among them, the STM32F103C8T6, as a representative model of the Cortex-M3 core, has been widely used in environmental monitoring, industrial control, and consumer electronics due to its excellent cost-performance ratio and broad application scenarios.

However, in the actual process of embedded system teaching and R&D, developers and learners face many challenges:

(1) **High hardware costs**. A complete environmental monitoring system requires the purchase of STM32 development boards, various sensor modules, debuggers, power supplies, and connecting cables, with a single set costing hundreds to thousands of yuan. For university laboratories, equipping dozens of complete experimental devices requires a large capital investment; for individual learners, the relatively high entry cost also forms a certain barrier.

(2) **Complex debugging process**. Embedded system debugging depends on the actual connection status of the hardware circuit. Any loose wire, wrong soldered resistor, or incorrect pin configuration may cause the system to fail to work properly. When facing complex hardware failures, beginners often find it difficult to quickly locate the root cause of the problem, consuming a lot of time on hardware troubleshooting rather than focusing on learning software algorithms and system logic.

(3) **High barrier to understanding sensors**. The working principles, communication protocols, and data processing methods of various sensors are different. For example, the DHT11 uses a single-wire protocol to transmit 40 bits of data, the MQ-135 needs to collect voltage through the ADC and infer gas concentration based on a complex mathe-

matical model, and the BMP280 involves I2C communication, 24-bit ADC reading, and temperature compensation algorithms. In-depth understanding of these sensors' working mechanisms requires learners to have knowledge reserves in analog circuits, digital circuits, communication protocols, and data processing.

(4) **Inconvenient data visualization**. In traditional embedded development environments, sensor data is usually output in text format through the serial port, and developers need to view raw values in a serial assistant, lacking intuitive data visualization means. Even with the help of host computer software, additional development work is often required, and the functions are relatively limited, making it difficult to achieve advanced functions such as remote Web access, historical data query, and intelligent alarms.

In summary, researching and implementing a virtualized intelligent environmental monitoring station based on STM32, which can simulate the complete embedded development process without real hardware, has important theoretical significance and practical value for lowering the learning threshold, improving teaching efficiency, and accelerating prototype verification.

1.2 Domestic and International Research Status

1.2.1 Research on Embedded Simulation Platforms

The research on embedded system virtualization and simulation technology has a long history, and domestic and foreign scholars and enterprises have carried out a lot of work in this field. QEMU (Quick Emulator) is an open-source general machine emulator and virtualizer that supports simulation of multiple processor architectures, including the ARM Cortex-M series. QEMU can efficiently run target platform binary code on the host machine through dynamic binary translation technology, and has been widely used in operating system development and embedded software testing. However, QEMU mainly focuses on simulation at the processor core and operating system level, and its support for simulation of specific microcontroller peripherals (such as GPIO, ADC, TIM, etc.) is relatively limited, and its configuration and usage threshold are high, making it less suitable for teaching and rapid prototype development scenarios.

Proteus is an electronic design automation (EDA) software developed by Labcenter Electronics in the UK, integrating circuit simulation, PCB design, and microcontroller co-simulation functions. Proteus supports simulation of the STM32 series microcontrollers and provides rich virtual instruments and oscilloscopes and other debugging tools, and is widely used in university electronic courses. However, Proteus is commercial software with high licensing fees, and its simulation speed is limited by the accuracy requirements of circuit-level simulation, resulting in low simulation efficiency for complex systems.

Domestically, universities such as Tsinghua University and Zhejiang University have

actively explored the construction of virtual experimental platforms for embedded systems. For example, some universities have developed Web-based remote embedded experimental platforms that allow students to remotely access real hardware devices located in laboratories through a browser for remote programming and debugging. Although such platforms achieve sharing of hardware resources, they still essentially rely on physical devices and do not fundamentally solve the problems of hardware cost and quantity limitations.

In recent years, with the popularization of high-level languages such as Python in education and scientific research, embedded teaching platforms based on pure software simulation have gradually attracted attention. Such platforms use high-level languages to simulate MCU core and peripheral behavior, with advantages such as high development efficiency, easy expansion, and cross-platform support, but there is a certain gap in simulation accuracy and execution efficiency compared with low-level emulators.

1.2.2 Research on Environmental Monitoring Systems

In the field of environmental monitoring, there are already a large number of mature products and solutions at home and abroad. The SHT series temperature and humidity sensors launched by Sensirion in the United States, the BMP series barometric pressure sensors from Bosch, and the CCS series gas sensors from AMS in Austria represent the high level of current environmental monitoring sensors. These sensors have characteristics such as high precision, low power consumption, and digital interfaces, and are widely used in professional environmental monitoring equipment.

China has also made significant progress in the research of environmental monitoring systems. Institutions such as the Chinese Academy of Sciences and Tsinghua University have conducted in-depth research on atmospheric pollution monitoring, water quality monitoring, and soil monitoring, and have developed multiple monitoring systems with independent intellectual property rights. In the consumer market, smart home environmental monitoring devices launched by companies such as Xiaomi and Huawei integrate multiple sensors and Wi-Fi communication modules, allowing users to view environmental data in real time through mobile apps, and have been widely welcomed by consumers.

In terms of IoT platforms, foreign platforms such as ThingSpeak and Adafruit IO, and domestic platforms such as OneNET and Alibaba Cloud IoT Platform, provide mature solutions for cloud storage, analysis, and visualization of environmental monitoring data. These platforms usually use the MQTT protocol to communicate with terminal devices, supporting rule engines and alarm notification functions.

However, the above research and products are mostly oriented toward actual application scenarios, paying less attention to the special needs in the teaching and learning process. How to virtualize and simplify the development process of environmental monitoring systems to make them suitable for embedded system teaching is still a problem

worth studying in depth.

1.3 Main Contributions of This Paper

This paper focuses on the virtualization of the intelligent environmental monitoring station based on STM32, and mainly completes the following four aspects of work:

(1) **Design and implementation of the virtual STM32 hardware platform**. Taking the STM32F103C8T6 as the prototype, a complete virtual MCU hardware platform is designed, including Cortex-M3 core simulation, memory mapping system, SysTick timer, GPIO general-purpose input/output, ADC analog-to-digital converter, I2C bus controller, UART serial communication, and TIM general-purpose timer and other core modules. The platform is implemented in Python, with advantages such as strong code readability, easy expansion, and cross-platform operation.

(2) **Development of virtual drivers for environmental sensors**. Based on the virtual hardware platform, virtual driver programs for five common environmental sensors, including DHT11, MQ-135, BH1750, BMP280, and noise sensors, are developed. A unified sensor modeling framework is established, including environmental parameter definitions, physical quantity conversion models, and Gaussian noise models, so that the simulated data conforms to physical laws and has a certain degree of randomness, which is closer to the output characteristics of real sensors.

(3) **Construction of the Web data visualization system**. The back-end service is developed using the Python Flask framework, the SQLite database is used to store sensor data, alarm records, and system logs, and RESTful API interfaces are designed for front-end calls. The front end uses native HTML/CSS/JavaScript technology, combined with the Chart.js chart library, to realize real-time line chart display of sensor data, statistical information summary, and alarm panel functions.

(4) **Environmental parameter control and interaction mechanism**. A file-sharing-based inter-process communication (IPC) mechanism is designed, and two-way communication between the Web control panel and the virtual MCU is achieved through the env_control.txt file. Users can adjust environmental parameters (such as temperature, humidity, air pressure, etc.) in real time through the Web interface, and the virtual sensors generate corresponding simulated data according to the updated parameters, achieving closed-loop interaction.

1.4 Paper Structure

This paper is divided into seven chapters, and the content of each chapter is arranged as follows:

Chapter 1 is the Introduction, introducing the research background and significance, domestic and international research status, main contributions of this paper, and the

structure of the paper.

Chapter 2 is the System Overall Design, analyzing the functional and non-functional requirements of the system, designing a four-layer overall architecture, and introducing the selection of key technologies.

Chapter 3 is the Virtual STM32 Hardware Platform Design, detailing the simulation principles and implementation methods of the virtual MCU core, memory system, and various peripherals.

Chapter 4 is Sensor Drivers and Data Acquisition, introducing the universal modeling framework for sensor virtualization, and analyzing in detail the simulation principles and driver implementations of each sensor.

Chapter 5 is Software System Design and Implementation, including the design and implementation of the data storage module, alarm system, Web visualization system, and control panel.

Chapter 6 is System Testing and Verification, conducting functional testing, performance testing, and sensor accuracy verification of the system, and giving an analysis of the test results.

Chapter 7 is Conclusion and Future Work, summarizing the main contributions of this paper, and proposing future improvement directions.

2 System Overall Design

2.1 Functional Requirements Analysis

2.1.1 Functional Requirements

This system aims to build a complete virtualized intelligent environmental monitoring station platform, and needs to meet the following functional requirements:

(1) **Virtual MCU Simulation**: The system should be able to simulate the core functions of the STM32F103C8T6 microcontroller, including processor instruction execution, memory access, clock system, and peripheral control. Embedded C code (or equivalent Python code) written by the user should be able to run correctly on the virtual platform and be migrated to real hardware without modification.

(2) **Multi-sensor Data Acquisition**: The system should support data acquisition from at least five environmental sensors, including temperature, humidity, air quality (expressed as harmful gas concentration), light intensity, atmospheric pressure, and noise level. Each sensor should have an independent driver program and follow the corresponding communication protocol.

(3) **Data Storage and Management**: The system should be able to persistently store the collected sensor data in a database, supporting query by time range, statistical

analysis, and data export functions. At the same time, it should record system operation logs and alarm events to facilitate troubleshooting and auditing.

(4) **Real-time Data Visualization**: The system should provide a real-time data display interface on the Web, intuitively displaying the current readings and historical trends of each sensor in the form of numeric cards and line charts. The charts should support time-axis zooming and data point tooltip functions.

(5) **Intelligent Alarm Function**: The system should support setting upper and lower threshold limits for each sensor. When a sensor reading exceeds the threshold range, an alarm is automatically triggered, and the alarm information is recorded in the database and front-end panel. The alarm status should be updated and cleared in real time.

(6) **Environmental Parameter Control**: The system should allow users to adjust environmental parameters in real time through the Web interface to simulate sensor behavior under different environmental conditions. Control commands should take effect in a timely manner, and the current environmental setting values should be fed back on the interface.

2.1.2 Non-functional Requirements

In addition to functional requirements, the system should also meet the following non-functional requirements:

(1) **Performance Requirements**: The instruction execution efficiency of the virtual MCU should be sufficiently high, and the processing time of a single data acquisition cycle should not exceed 1 second. The Web API response time should be controlled within 100 milliseconds, and the refresh frequency of the front-end charts should not be lower than 1 Hz.

(2) **Reliability Requirements**: The system should be able to run continuously and stably for at least 72 hours without crashing. Database operations should have transaction protection mechanisms to prevent data loss or inconsistency. Sensor drivers should be able to handle abnormal situations such as communication timeouts and checksum errors.

(3) **Scalability Requirements**: The system architecture should have a good modular design. When adding new sensor types or peripheral modules, it should be achievable by adding driver files and configuration files without modifying the core code. The Web back-end API design should follow RESTful specifications to facilitate the expansion of front-end functions.

(4) **Usability Requirements**: The system should provide clear documentation and instructions, and the Web interface should have a reasonable layout and intuitive operation. The virtual hardware API interface should be designed to be simple, reducing the user's learning cost and development difficulty.

(5) **Cross-platform Requirements**: The system should be able to run on main-

stream operating systems such as Windows, Linux, and macOS, without relying on specific hardware or operating system features. The virtual platform itself should be implemented in Python, utilizing Python's cross-platform characteristics to ensure compatibility.

2.2 Overall Architecture Design

Based on the above requirements analysis, this system adopts a four-layer architecture design, from bottom to top: Hardware Abstraction Layer, Device Driver Layer, Business Logic Layer, and Application Presentation Layer. The responsibilities of each layer are shown in Table 1.

Table 1: System Four-Layer Architecture Design

Layer	**Name**	**Responsibility Description**
Layer 4	Application Presentation Layer	Responsible for Web front-end interface rendering, chart drawing, user interaction, and data display, implemented using native HTML/CSS/JavaScript
Layer 3	Business Logic Layer	Responsible for Flask back-end services, RESTful API design, data storage management, alarm logic processing, and control command parsing
Layer 2	Device Driver Layer	Responsible for the development of various sensor driver programs and the implementation of the data acquisition loop, converting physical quantities into digital quantities
Layer 1	Hardware Abstraction Layer	Responsible for the simulation of the virtual MCU core, memory system, bus controllers, and various peripherals, providing a hardware register-level interface

The Hardware Abstraction Layer (HAL) is the foundation of the entire system, providing a software interface consistent with the behavior of real STM32 hardware registers. The Device Driver Layer, based on the interface provided by the HAL, implements the initialization, data reading, and protocol parsing functions of various sensors. The Business Logic Layer is responsible for processing, storing, and analyzing raw sensor data, and providing standardized APIs for upper-layer calls. The Application Presentation Layer focuses on the presentation of the user interface and the optimization of the interaction experience.

The four-layer architecture communicates through well-defined interfaces. Upper-layer modules do not directly access the internal implementation of lower-layer modules, thus ensuring the loose coupling and maintainability of the system. When a new sensor type needs to be added, only the corresponding driver file needs to be added in the Device Driver Layer, and data parsing rules need to be registered in the Business Logic Layer, without modifying the code of the Hardware Abstraction Layer and the Application Presentation Layer.

2.3 Technology Selection

The technology selection of this system follows the principles of "simple, efficient, and easy to expand". The main technology stack is shown in Table 2.

Table 2: System Technology Selection

Category	**Technology**	**Selection Reason**
Development Language	Python 3.11	Concise syntax, high development efficiency, rich standard library and third-party libraries, good cross-platform support
Web Framework	Flask 3.0	Lightweight, flexible, easy to learn, active community, rich extensions
Database	SQLite 3	Zero configuration, single-file storage, no independent server process required, suitable for small and medium-sized applications
Front-end Technology	Native HTML/CSS/JS	No build dependencies, fast loading, good compatibility, easy to deploy
Chart Library	Chart.js 4.x	Open source and free, friendly API, supports responsive design and animation effects
Version Control	Git	Distributed version management, flexible branch management, high collaboration efficiency

Choosing Python as the core development language is mainly based on the following considerations: First, Python's syntax is concise and clear, with strong code readability, making it very suitable for code display and explanation in teaching scenarios; Second, Python has a rich standard library, covering commonly used functions such as network communication, database access, file operations, and multi-threaded programming, with-

out the need to install a large number of additional dependencies; Third, Python's interpreted execution feature makes the development and debugging cycle short, and code can be run without compilation after modification, improving development efficiency.

Flask is a lightweight Web framework written in Python. Its core design philosophy is to keep it simple and flexible. Compared with full-featured frameworks such as Django, Flask has no mandatory project structure or component dependencies. Developers can freely choose databases, template engines, and authentication solutions according to actual needs. For Web applications of medium complexity like this system, Flask provides just the right functional support, neither too cumbersome nor insufficient.

SQLite is an embedded relational database. The entire database is stored in a single disk file, without the need for a separate database server process. This "zero configuration" feature makes system deployment extremely simple, especially suitable for personal development, teaching experiments, and small application scenarios. SQLite supports standard SQL syntax and transaction mechanisms, which can meet the system's needs for data persistence and query analysis.

In terms of front-end technology, this system is developed using native HTML5, CSS3, and JavaScript, without introducing front-end frameworks such as React or Vue. The main purpose of this is to reduce system complexity, reduce the configuration work of build tools, and make the front-end code more intuitive and easy to understand. Chart.js is an excellent open-source chart library, providing rich chart types and configuration options. Its Canvas-based rendering method has good performance and can meet the needs of real-time data visualization.

3 Virtual STM32 Hardware Platform Design

The virtual STM32 hardware platform is the core foundation of this system. Its design goal is to accurately simulate the hardware behavior of the STM32F103C8T6 microcontroller at the software level, providing upper-layer sensor drivers and applications with a programming interface consistent with real hardware. This chapter will elaborate on the design principles and implementation methods of the platform from three aspects: virtual MCU core design, memory system design, and peripheral simulation.

3.1 Virtual MCU Core Design

3.1.1 STM32F103C8T6 Parameter Overview

The STM32F103C8T6 is a 32-bit microcontroller based on the ARM Cortex-M3 core launched by STMicroelectronics, belonging to the "medium-density" product line of the STM32F1 series. This model has become one of the most popular microcontrollers in

the embedded development field due to its excellent performance and low price. Its main technical parameters are shown in Table 3.

Table 3: STM32F103C8T6 Main Technical Parameters

Parameter Item	**Specification**
Core Architecture	ARM Cortex-M3
Maximum Frequency	72 MHz
Flash Memory	64 KB
SRAM	20 KB
GPIO Pins	37 (LQFP48 package)
ADC	2 12-bit ADCs, 16 channels
Timers	3 general-purpose timers + 1 advanced timer + 2 basic timers
Communication Interfaces	2 I2C, 3 USART, 2 SPI, 1 CAN
Operating Voltage	2.0 V – 3.6 V
Operating Temperature	−40°C – +85°C
Package Type	LQFP48

In the virtual platform, since it is implemented in the high-level Python language, it does not involve actual instruction set simulation and clock-cycle-accurate simulation. Therefore, the focus is mainly on the simulation of peripheral register-level behavior. The virtual MCU uses 1 millisecond as the minimum time granularity, and implements delay and time management functions through Python's time module.

3.1.2 SysTick Timer Simulation

SysTick is a 24-bit down-counter built into the Cortex-M3 core, providing periodic timer interrupts for the operating system and applications. In the virtual platform, the simulation of SysTick mainly includes three functions: counter decrement, reload value setting, and interrupt triggering. Since Python is an interpreted language, it is impossible to accurately simulate microsecond-level clock cycles, so an approximate simulation using relative time measurement is adopted.

The core function of the SysTick timer is implemented by the delay_ms function, which provides millisecond-level delay capability for upper-layer applications. Its working principle is as follows: record the system timestamp when the function starts, and then continuously check the difference between the current time and the start time in a loop. When the difference reaches or exceeds the target number of milliseconds, the loop exits. Although this implementation consumes CPU resources, it is sufficient for teaching demonstrations and functional verification scenarios.

```
1 import time
```

```
2
3  def delay_ms(ms):
4      """
5      Millisecond-level delay function
6      Parameters:
7          ms: delay milliseconds
8      """
9      start = time.time()
10     while (time.time() - start) * 1000 < ms:
11         pass
```

Listing 1: Virtual delay_ms function implementation

As shown in Code 1, the delay_ms function implements delay through busy-waiting. In actual teaching, students can be guided to think about the advantages and disadvantages of this implementation: the advantage is that the code is simple and does not require external libraries; the disadvantage is that the CPU is completely occupied during the delay and cannot execute other tasks. As an improvement, Python's threading module or asyncio module can be introduced to achieve non-blocking delay.

3.2 Memory System Design

3.2.1 Memory Mapping Model

The STM32F103C8T6 adopts a unified 32-bit address space, with various memories and peripheral registers mapped to different regions according to fixed addresses. The memory system of the virtual platform needs to simulate the following key regions:

(1) **Flash Memory**: Used to store program code and constant data, with the address range starting from 0x08000000 and a size of 64 KB. In the virtual platform, the Flash region is mainly used to store simulated firmware code and configuration parameters.

(2) **SRAM**: Used to store runtime variables, stack, and heap memory, with the address range starting from 0x20000000 and a size of 20 KB. The SRAM region in the virtual platform is implemented as a byte array, supporting byte, half-word, and word read/write access.

(3) **Peripheral Registers**: Including configuration registers and data registers for peripherals such as GPIO, ADC, UART, TIM, and I2C, with the address range starting from 0x40000000. Each peripheral occupies a continuous address space, and registers are arranged according to fixed offsets.

The core class of the virtual memory system is MemoryRegion, which is responsible for managing read and write operations in a continuous address space. MemoryRegion supports endianness control. As a little-endian processor, the STM32 stores the low byte

of multi-byte data at the low address and the high byte at the high address.

3.2.2 Little-Endian Implementation

Little-Endian means that the most significant byte of data is stored at the high address end of memory, and the least significant byte is stored at the low address end. For example, the storage order of the 32-bit data 0x12345678 in memory is: address 0x00 stores 0x78, address 0x01 stores 0x56, address 0x02 stores 0x34, and address 0x03 stores 0x12.

To implement little-endian data access in Python, the formatting string function of the struct module needs to be used. The struct.pack and struct.unpack functions specify little-endian mode with the format character '<' and big-endian mode with '>'. The MemoryRegion class automatically performs endianness conversion when reading and writing multi-byte data, ensuring consistency with the behavior of a real STM32.

3.2.3 Register Mapping

The RegisterMap class is a further encapsulation of MemoryRegion, specially designed for peripheral register management. Each peripheral instance has a RegisterMap object, which accesses registers by name rather than absolute address, improving code readability and maintainability.

The core functions of RegisterMap include:

(1) **Register Registration**: Define all registers owned by the peripheral during initialization, including register name, offset address, default value, and access permissions (read-only/write-only/read-write).

(2) **Bit-field Access**: Supports read and write operations on individual bits or bit-fields within a register. For example, set_bit('CR1', 0) can be used to set bit 0 of the CR1 register to 1 without first reading the entire register and manually calculating.

(3) **Write Protection**: For read-only registers (such as the status register SR), RegisterMap will reject direct write operations to prevent upper-layer code from accidentally modifying register values that should not be modified.

(4) **Change Callback**: Allows registering callback functions for specific registers, which are automatically triggered when the register value changes. This mechanism is very important for simulating peripheral behavior responses. For example, when the ODR register of GPIO is written, the output state of the pin needs to be updated immediately.

3.3 Peripheral Simulation

3.3.1 GPIO General Purpose Input/Output

GPIO (General Purpose Input/Output) is the most basic and important peripheral of the microcontroller, responsible for digital signal input and output with external devices. The GPIO module of the STM32F103C8T6 has the following characteristics: each GPIO

port contains 16 pins (numbered 0–15), supporting input modes (floating input, pull-up input, pull-down input, analog input) and output modes (push-pull output, open-drain output), and other configurations.

In the virtual platform, GPIO simulation mainly includes the following components:

(1) **Pin State Array**: Each GPIO port maintains an array of length 16, recording the current logic level state (0 or 1) of each pin. For input mode, the pin state is determined by external devices or pull-up/pull-down resistors; for output mode, the pin state is determined by the value of the ODR register.

(2) **IDR Input Data Register**: IDR (Input Data Register) is used to read the current input logic level of each pin of the GPIO port. In the virtual platform, the value of IDR is obtained by encoding the pin state array. Specifically, the i-th bit of IDR equals the current logic level state of pin i.

The calculation formula for IDR is:

$$\text{IDR} = \sum_{i=0}^{15} P_i \times 2^i \tag{1}$$

where P_i represents the logic level state of pin i, $P_i \in \{0, 1\}$.

(3) **ODR Output Data Register**: ODR (Output Data Register) is used to set the output logic level of each pin of the GPIO port. When a new value is written to the ODR, the virtual GPIO module parses it bit by bit, updates the output state of the corresponding pin, and triggers the registered callback functions.

(4) **Callback Mechanism**: To support the response of external devices to GPIO state changes, the virtual GPIO implements a callback registration mechanism. Users can register callback functions for rising edge, falling edge, or level change events on specific pins. When the pin state changes accordingly, all registered callback functions are executed in the order of registration. This mechanism is very useful when simulating sensor interrupt outputs and LED indicator on/off control.

3.3.2 ADC Analog-to-Digital Converter

ADC (Analog-to-Digital Converter) is responsible for converting analog voltage signals into digital quantities, and is the bridge connecting analog sensors and digital systems. The STM32F103C8T6 has two built-in 12-bit successive approximation ADCs, supporting up to 16 external input channels and 2 internal channels (temperature sensor and internal reference voltage).

The core parameters and characteristics of the virtual ADC include:

(1) **12-bit Resolution**: The conversion result of the ADC is a 12-bit unsigned integer,

with a value range of 0 to 4095. The corresponding quantization formula is:

$$D = \left\lfloor \frac{V_{in}}{V_{ref}} \times 4096 \right\rfloor \tag{2}$$

where D is the digital output, V_{in} is the input analog voltage, and V_{ref} is the reference voltage (usually 3.3 V).

(2) **Voltage-to-Digital Conversion**: When upper-layer code reads a channel through the ADC, the virtual ADC calculates the corresponding digital quantity according to the current analog voltage value connected to that channel based on Formula 2. The analog voltage value is provided by the connected virtual sensor or external signal source.

(3) **Noise Model**: To more realistically simulate the conversion characteristics of the ADC, Gaussian white noise is superimposed on the ideal conversion result. The amplitude of the noise can be adjusted through configuration parameters, with a default of ±2 LSB (Least Significant Bit). The conversion formula with noise is:

$$D_{actual} = D_{ideal} + N(0, \sigma^2) \tag{3}$$

where $N(0, \sigma^2)$ represents a normally distributed random number with mean 0 and variance σ^2, and the default value of σ is approximately 2.

(4) **Channel Selection and Enable**: The virtual ADC supports multi-channel configuration, defining the order and number of conversion channels through the SQR (Regular Sequence Register) sequence register. Before performing a conversion, the ADC needs to be enabled first (set the ADON bit of the CR2 register to 1), and then the conversion is started (set the SWSTART bit of the CR2 register to 1).

3.3.3 I2C Bus Controller

I2C (Inter-Integrated Circuit) is a widely used serial communication bus protocol that enables multi-master multi-slave communication with only two signal lines (SDA data line and SCL clock line). The STM32F103C8T6 has two built-in I2C interfaces, supporting standard mode (100 kHz) and fast mode (400 kHz).

The simulation focus of the virtual I2C module includes:

(1) **Device Attachment**: The virtual I2C bus maintains a list of attached devices, each uniquely identified by a 7-bit slave address. When upper-layer code sends a start signal and address byte through I2C, the bus controller looks up the corresponding virtual device instance based on the address and establishes a communication connection.

The interface definition for device attachment is as follows:

```
def attach_device(self, address, device):
    """
    Attach a virtual device to the I2C bus
```

```
    Parameters:
        address: 7-bit slave address
        device:  virtual device instance, must implement read/write
           methods
    """
    self.devices[address] = device
```

Listing 2: Virtual I2C device attachment interface

(2) **Read and Write Operations**: The virtual I2C supports byte-level and buffer-level read and write operations. During a write operation, the master device sends the slave address and write flag bit (0), followed by the register address to be written and the data bytes; during a read operation, the master device first sends the slave address and write flag bit to specify the register address, then resends the start signal and the slave address with read flag bit (1), and the slave device returns the requested data bytes.

(3) **Error Handling**: The virtual I2C implements basic error detection mechanisms, including: address no-acknowledge (NACK) — returns an error when the sent slave address cannot be found in the device list; bus busy — generates conflict detection when multiple master devices attempt to control the bus simultaneously; data verification — supports CRC verification of transmitted data (optional).

3.3.4 UART Serial Communication

UART (Universal Asynchronous Receiver/Transmitter) is an asynchronous serial communication protocol widely used for data transmission between microcontrollers and PCs, Bluetooth modules, GPS modules, and other devices. The STM32F103C8T6 provides three independent USART interfaces, supporting full-duplex communication and multiple baud rate configurations.

The design points of the virtual UART module include:

(1) **Baud Rate Configuration**: The baud rate represents the number of bits transmitted per second and is a key parameter of UART communication. The virtual UART records the currently configured baud rate value. Although due to the nature of software simulation it is impossible to truly perform bit-level timing control according to the baud rate, the baud rate value will affect certain timing-related simulation behaviors (such as timeout settings). Common baud rate values include 9600, 115200, 921600, etc.

(2) **printf Redirection Simulation**: In embedded development, the standard output function printf is usually redirected to the UART to output debugging information through the serial port. The virtual UART provides an equivalent string sending interface, caching the string to be sent into an internal buffer and triggering the receiver-side callback function.

(3) **Callback Mechanism**: The virtual UART supports registering receive callback

functions, which are automatically called when data arrives. This mechanism simulates the receive interrupt of a real UART, allowing upper-layer code to process serial data in an event-driven manner without polling the receive buffer.

```
class VirtualUART:
    def __init__(self):
        self.baudrate = 115200
        self.rx_callback = None
        self.tx_buffer = []

    def set_rx_callback(self, callback):
        """Register receive callback function"""
        self.rx_callback = callback

    def send(self, data):
        """Send data (string or bytes)"""
        if isinstance(data, str):
            data = data.encode('utf-8')
        self.tx_buffer.extend(data)
        if self.rx_callback:
            self.rx_callback(data)

    def printf(self, fmt, *args):
        """Formatted output, simulating printf"""
        msg = fmt % args
        self.send(msg)
```

Listing 3: Virtual UART callback registration and sending

As shown in Code 3, the VirtualUART class provides a concise interface for data sending and callback registration. The send method accepts string or byte type data, and the printf method simulates the formatted output function of the C language printf function.

3.3.5 TIM General Purpose Timer

TIM (Timer) is one of the most functionally rich peripherals in the STM32, and can be used as a timer, counter, PWM generator, and input capture, etc. This system mainly uses the basic timing function of TIM to provide periodic trigger signals for sensor data acquisition.

The implementation of the virtual TIM module is based on Python's threading module, using a daemon thread running in the background. The characteristic of a daemon thread is that when the main thread ends, the daemon thread will automatically terminate

without explicit shutdown, making it suitable for periodic background tasks.

The core workflow of the virtual TIM is as follows:

(1) **Counter Increment**: After TIM is started, the daemon thread increments the internal counter at a fixed time interval (determined by the prescaler coefficient and auto-reload value). The counter increment frequency calculation formula is:

$$f_{count} = \frac{f_{clk}}{(PSC + 1) \times (ARR + 1)} \tag{4}$$

where f_{clk} is the timer clock frequency (usually 72 MHz), PSC is the prescaler value, and ARR is the auto-reload value.

(2) **Periodic Callback**: When the counter reaches the auto-reload value, the counter is reset to zero and an update event is triggered. If a periodic callback function has been registered, it is called at this time. In the environmental monitoring system, the periodic callback function usually performs sensor data acquisition tasks, thereby achieving automatic sampling at a fixed frequency.

(3) **Start and Stop**: TIM controls start and stop through the CEN bit of the CR1 register. Setting the CEN bit to 1 starts the timer, and clearing it stops the timer. The virtual TIM creates and starts a daemon thread when starting, and sets an event flag to notify the thread to exit when stopping.

```
import threading
import time

class VirtualTIM:
    def __init__(self):
        self.prescaler = 7199       # PSC
        self.auto_reload = 9999     # ARR
        self.counter = 0
        self.running = False
        self.period_callback = None
        self._stop_event = threading.Event()

    def set_period_callback(self, callback):
        """Set periodic callback function"""
        self.period_callback = callback

    def _timer_thread(self):
        """Timer daemon thread"""
        period_ms = (self.prescaler + 1) * (self.auto_reload + 1) /
            72000
        while not self._stop_event.is_set():
```

```
            time.sleep(period_ms / 1000.0)
            self.counter = 0
            if self.period_callback:
                self.period_callback()

    def start(self):
        """Start timer"""
        self.running = True
        self._stop_event.clear()
        t = threading.Thread(target=self._timer_thread, daemon=True)
        t.start()

    def stop(self):
        """Stop timer"""
        self.running = False
        self._stop_event.set()
```

Listing 4: Virtual TIM daemon thread implementation

As shown in Code 4, the VirtualTIM class implements periodic timing functions through a daemon thread. period_callback is the callback function registered by the user, which is called at the end of each timing period. By adjusting the values of prescaler and auto_reload, the timing period can be flexibly configured to adapt to different sampling frequency requirements.

4 Sensor Drivers and Data Acquisition

Sensors are the "senses" of the environmental monitoring system, responsible for converting environmental parameters in the physical world into electrical signals, which are then processed and analyzed by the microcontroller. In the virtualization platform, sensor simulation is the key bridge connecting virtual hardware and upper-layer applications, and its simulation accuracy directly determines the credibility of the entire system. This chapter first introduces the universal modeling framework for sensor virtualization, and then elaborates on the simulation principles and driver implementations of each sensor.

4.1 Sensor Virtualization Principles

4.1.1 Universal Modeling Framework

In order to achieve consistency and scalability of sensor simulation, this system defines a universal sensor modeling framework. This framework abstracts a sensor into three core

components: environmental parameter input, physical quantity conversion model, and digital quantity output. Its workflow is shown in Figure **??** (conceptual description).

The core elements of the universal modeling framework include:

(1) **Environmental Parameter Definition**: Each sensor is associated with a set of environmental parameter variables, which represent the true values of the physical quantities measured by the sensor. For example, the temperature sensor is associated with the temperature parameter, and the light sensor is associated with the illuminance parameter. The values of environmental parameters can be adjusted in real time through the control panel to simulate different environmental conditions.

(2) **Physical Quantity Conversion Model**: Describes how environmental parameters are converted into the physical quantities output by the sensor (usually voltage, resistance, or digital quantity). The conversion models of different sensors vary, and may be linear, power-law, or piecewise function relationships. The conversion model should be established based on the sensor's datasheet and physical principles to ensure the rationality of simulated data.

(3) **Noise Model**: The output of real sensors inevitably contains random noise, the sources of which include thermal noise, shot noise, and electromagnetic interference, etc. To enhance the authenticity of the simulation, Gaussian white noise is superimposed on the ideal output of the sensor. The mathematical expression of the noise model is:

$$Y = X + N(0, \sigma^2) \tag{5}$$

where Y is the actual output value of the sensor, X is the ideal output value (calculated by the physical quantity conversion model), and $N(0, \sigma^2)$ is random noise following a normal distribution with mean 0 and variance σ^2.

The value of the noise intensity σ is determined according to the sensor's accuracy grade and typical application scenarios. For high-precision sensors (such as BMP280), σ takes a smaller value; for low-precision sensors (such as MQ-135), σ takes a larger value. In special scenarios, outliers can also be added to simulate sensor failures or sudden environmental changes.

4.1.2 Sensor Driver Interface

All virtual sensor drivers follow a unified interface specification, facilitating upper-layer code to access different types of sensors in a consistent manner. The core interfaces include:

- `init()`: Initialize the sensor, configure communication parameters and working modes.
- `read()`: Read the current measured value of the sensor, returning raw data or calibrated physical quantities.

- `calibrate()`: Perform calibration operations, update calibration coefficients (if applicable).
- `set_environment(env_dict)`: Update the environmental parameters associated with the sensor.

The unified interface design allows new sensor types to be seamlessly integrated into the existing system by simply implementing the above interfaces, without modifying the data acquisition loop and Web display layer code.

4.2 DHT11 Temperature and Humidity Sensor

4.2.1 Sensor Overview

The DHT11 is a digital sensor integrating temperature measurement and humidity measurement functions, using a single-wire (One-Wire) serial communication protocol. This sensor is inexpensive, compact, and easy to use, and is one of the most commonly used temperature and humidity sensors in entry-level IoT projects. Its main technical parameters are shown in Table 4.

Table 4: DHT11 Main Technical Parameters

Parameter Item	**Specification**
Measurement Range (Humidity)	20% – 90% RH
Measurement Accuracy (Humidity)	±5% RH
Measurement Range (Temperature)	0°C – 50°C
Measurement Accuracy (Temperature)	±2°C
Resolution	1% RH / 1°C
Sampling Period	$\geq$ 1 s
Supply Voltage	3.3 V – 5.5 V
Communication Protocol	Single-wire serial

4.2.2 One-Wire Protocol Simulation

The single-wire communication protocol of the DHT11 is divided into three stages: host start signal, DHT11 response signal, and data transmission.

(1) **Host Start Signal**: The host (MCU) pulls the data line low for at least 18 ms, then releases the data line and waits for the DHT11 to respond. The purpose of this stage is to wake up the DHT11 sensor in a low-power state and notify it to prepare to send data.

(2) **DHT11 Response Signal**: After detecting the start signal, the DHT11 first pulls the data line low for 80 microseconds and then high for 80 microseconds, indicating

that it is ready to send data. The host confirms that the DHT11 is online and working normally by detecting this response signal.

(3) **Data Transmission**: The DHT11 sends temperature and humidity information in the format of a 40-bit data frame. Each bit of data starts with a 50-microsecond low level, and the duration of the subsequent high level determines the value of the bit: a high level lasting 26–28 microseconds represents "0", and lasting 70 microseconds represents "1". The 40-bit data includes: 8-bit humidity integer, 8-bit humidity decimal, 8-bit temperature integer, 8-bit temperature decimal, and 8-bit checksum.

The virtual DHT11 driver implements a complete simulation of the above protocol. The core code is as follows:

```
class VirtualDHT11:
    def __init__(self, gpio, pin):
        self.gpio = gpio
        self.pin = pin
        self.humidity = 50.0    # Default humidity 50%
        self.temperature = 25.0 # Default temperature 25C

    def set_environment(self, env):
        self.humidity = env.get('humidity', 50.0)
        self.temperature = env.get('temperature', 25.0)

    def read(self):
        # Simulate start signal
        self.gpio.set_mode(self.pin, 'OUTPUT')
        self.gpio.write(self.pin, 0)
        delay_ms(20)
        self.gpio.write(self.pin, 1)
        delay_us(30)
        self.gpio.set_mode(self.pin, 'INPUT')

        # Simulate response signal and data transmission
        # Returns (humidity, temperature)
        h_int = int(self.humidity)
        h_dec = int((self.humidity - h_int) * 10)
        t_int = int(self.temperature)
        t_dec = int((self.temperature - t_int) * 10)
        checksum = (h_int + h_dec + t_int + t_dec) & 0xFF

        data = [h_int, h_dec, t_int, t_dec, checksum]
        return self.humidity + random.gauss(0, 1.5), \
               self.temperature + random.gauss(0, 0.8)
```

Listing 5: DHT11 start signal and data reading

As shown in Code 5, the virtual DHT11 driver simulates the key timing of the single-wire protocol by controlling the GPIO pin levels. In the data generation stage, environmental parameters are converted into a 40-bit data format, and the checksum is calculated. At the same time, Gaussian noise is superimposed according to Formula 5, with the humidity noise standard deviation set to 1.5% RH and the temperature noise standard deviation set to 0.8°C, matching the actual accuracy grade of the DHT11.

4.3 MQ-135 Air Quality Sensor

4.3.1 Sensor Principle

The MQ-135 is a broad-spectrum gas sensor with high sensitivity to ammonia, sulfides, benzene vapors, smoke, and harmful gases. Its sensitive element is made of tin dioxide (SnO_2) semiconductor material. When the target gas is present in the environment, gas molecules undergo redox reactions on the surface of the sensitive material, causing the resistance value of the material to change. By measuring the change in resistance, the gas concentration can be indirectly estimated.

The relationship between the sensitive element resistance R_S of the MQ-135 and the gas concentration C follows a power-law relationship:

$$\frac{R_S}{R_0} = A \cdot C^{-B} \tag{6}$$

where R_0 is the reference resistance value of the sensor in clean air, A and B are empirical constants related to the gas type, and C is the gas concentration (unit: ppm). For general air quality monitoring scenarios, $A \approx 116.6$ and $B \approx 2.77$ are usually taken.

4.3.2 Voltage-to-Concentration Inversion

MQ-135 sensor modules usually use a simple voltage divider circuit to convert resistance changes into voltage output. The relationship between the output voltage V_{out} of the voltage divider circuit and the sensitive element resistance R_S is:

$$V_{out} = V_{cc} \cdot \frac{R_L}{R_S + R_L} \tag{7}$$

where V_{cc} is the supply voltage (usually 5 V) and R_L is the load resistance (adjustable potentiometer).

From this, the sensitive element resistance can be inversely derived:

$$R_S = R_L \cdot \left(\frac{V_{cc}}{V_{out}} - 1 \right) \tag{8}$$

Further substituting into Formula 6, the gas concentration can be obtained:

$$C = \left(\frac{A \cdot R_0}{R_S} \right)^{\frac{1}{B}} = \left(\frac{A \cdot R_0 \cdot V_{out}}{R_L \cdot (V_{cc} - V_{out})} \right)^{\frac{1}{B}} \tag{9}$$

4.3.3 Virtual Driver Implementation

The core task of the virtual MQ-135 driver is to calculate the corresponding ADC voltage value according to the air_quality value (unit: ppm) in the environmental parameters, based on Formulas 6 and 9. In order to simulate the behavior of a real sensor, the following characteristics are also implemented:

(1) **Warm-up Simulation**: The MQ-135 sensor requires a period of warm-up after power-on to reach a stable working state, with a typical warm-up time of 20 seconds. The virtual driver records the start time after initialization, and returns invalid data or lower-precision data during the warm-up period.

(2) **Non-linear Response**: According to the power-law relationship, the relationship between gas concentration and resistance value is non-linear. In the low concentration region, the sensor has higher sensitivity; in the high concentration region, the sensitivity gradually decreases. The virtual driver fully simulates this non-linear characteristic.

(3) **Temperature and Humidity Compensation**: The output of the MQ-135 is affected by ambient temperature and humidity. The virtual driver introduces a simple temperature and humidity compensation coefficient, which corrects the concentration value according to the current temperature and humidity, improving the authenticity of the simulated data.

4.4 BH1750 Light Sensor

4.4.1 Sensor Overview

The BH1750 is a digital light intensity sensor using the I2C communication interface, which can directly output illuminance values in lux without complex analog-to-digital conversion and calibration calculations. This sensor has a wide range, high accuracy, and is easy to use, and is widely used in automatic lighting control, backlight adjustment, and environmental monitoring.

The BH1750 supports multiple measurement modes, which are switched by sending different command bytes to the sensor. Common commands are shown in Table 5.

Table 5: BH1750 Common I2C Commands

Command Value	Name	Function Description
0x01	Power On	Power-on command, wakes up the sensor
0x00	Power Down	Power-down command, enters low-power mode
0x10	Continuous H-Res Mode	Continuous high-resolution mode (1 lux resolution, 120 ms measurement time)
0x11	Continuous H-Res2 Mode	Continuous high-resolution mode 2 (0.5 lux resolution, 120 ms measurement time)
0x13	Continuous L-Res Mode	Continuous low-resolution mode (4 lux resolution, 16 ms measurement time)
0x20	One Time H-Res Mode	One-time high-resolution mode

4.4.2 Illuminance Calculation Formula

The output value of the BH1750 in high-resolution mode (H-Res Mode) is 16-bit raw data, which needs to be multiplied by a sensitivity coefficient to obtain the illuminance in lux. The standard sensitivity coefficient is 1.2 lux/count, and the calculation formula is:

$$E_v = \text{raw} \times 1.2 \tag{10}$$

where E_v is the illuminance (unit: lux), and raw is the 16-bit raw data returned by the sensor.

For example, when the raw data is 500, the corresponding illuminance is $500 \times 1.2 = 600$ lux.

The virtual BH1750 driver communicates with the MCU through the I2C interface. After receiving the command byte, it inversely derives the raw data according to the illuminance value in the current environmental parameters based on Formula 10:

$$\text{raw} = \left\lfloor \frac{E_v}{1.2} \right\rfloor + N(0, \sigma^2) \tag{11}$$

where σ takes 3 count units, corresponding to approximately 3.6 lux measurement noise, which is consistent with the BH1750 datasheet specification.

4.5 BMP280 Barometric Pressure Sensor

4.5.1 Sensor Overview

The BMP280 is a high-precision barometric pressure and temperature sensor launched by Bosch, manufactured using MEMS technology, with advantages such as small size, low power consumption, and high accuracy. This sensor is widely used in meteorological monitoring, altitude measurement, indoor navigation, and drone altitude control. Its main technical parameters are shown in Table 6.

Table 6: BMP280 Main Technical Parameters

Parameter Item	**Specification**
Pressure Measurement Range	300 hPa – 1100 hPa
Pressure Measurement Accuracy	±0.12 hPa (typical)
Temperature Measurement Range	−40°C – +85°C
Temperature Measurement Accuracy	±0.5°C (at 25°C)
Absolute Accuracy	±1 hPa
Communication Interface	I2C or SPI
Chip ID	0x58

4.5.2 Chip ID Verification

The BMP280 has a read-only chip ID register at address 0xD0, with a default value of 0x58 after power-on. Reading this register is a common method to verify whether the sensor is correctly connected and working. The virtual BMP280 driver simulates this behavior during initialization, returning 0x58 when the host reads the 0xD0 register.

4.5.3 24-bit ADC and Calibration Coefficients

The BMP280 integrates a 24-bit ADC to convert the analog signals output by the barometric pressure and temperature sensor elements into digital quantities. The barometric pressure ADC output is 20-bit effective data (stored in 3 bytes), and the temperature ADC output is also 20-bit effective data. The high resolution ensures that the sensor can resolve extremely small barometric pressure changes.

The real BMP280 is factory-programmed with a set of calibration coefficients stored in NVM at addresses 0x88–0xA1. Before reading measurement data, the host needs to read these calibration coefficients, and then perform compensation algorithms in software to convert raw ADC values into compensated physical quantities. The compensation algorithm formulas are relatively complex, involving quadratic and cubic polynomial calculations.

In the virtual platform, in order to simplify the implementation while maintaining

the rationality of the data, a simplified calibration model is adopted. The mapping relationship between raw ADC values and environmental parameters is approximated using a linear formula:

$$\text{raw\_P} = \left\lfloor \frac{P}{1013.25} \times 415148 \right\rfloor + N(0, \sigma_P^2) \tag{12}$$

$$\text{raw\_T} = \lfloor (T + 40) \times 100 \rfloor + N(0, \sigma_T^2) \tag{13}$$

where P is the environmental pressure value (unit: hPa), T is the environmental temperature value (unit: °C), 415148 is the typical raw ADC value corresponding to 1013.25 hPa (standard atmospheric pressure), and σ_P and σ_T are noise standard deviations.

The derivation basis of Formula 12 is: the raw ADC value of the BMP280 near standard atmospheric pressure is approximately 415148 (corresponding to oversampling ×16 mode), so a proportional relationship between the pressure value and the raw value is established. Although the complex temperature compensation and non-linear calibration processes of the real BMP280 are simplified, it is sufficiently accurate for teaching demonstrations and general verification.

4.6 Noise Sensor

4.6.1 Sensor Principle

The noise sensor (sound sensor) is used to measure the intensity of environmental noise, usually expressed in decibels (dB). Common noise sensor modules (such as KY-038 or LM393-based modules) integrate a high-sensitivity microphone and signal conditioning circuit, converting sound signals into analog voltage output.

The decibel is a logarithmic unit used to represent the ratio of two power values or sound pressure values. The definition formula for Sound Pressure Level (SPL) is:

$$L_p = 20 \cdot \log_{10} \left(\frac{p}{p_0} \right) \tag{14}$$

where L_p is the sound pressure level (unit: dB), p is the actual sound pressure (unit: Pa), and p_0 is the reference sound pressure (usually 20×10^{-6} Pa, i.e., the hearing threshold of the human ear at a frequency of 1 kHz).

4.6.2 Linear Voltage Mapping

Noise sensor modules usually output analog voltage signals proportional to the environmental sound pressure (after amplification and rectification). In order to simplify the virtual model, it is assumed that there is a linear mapping relationship between the sen-

sor output voltage and the sound pressure level:

$$V_{out} = V_{offset} + k \cdot L_p \tag{15}$$

where V_{offset} is the offset voltage (corresponding to the output at 0 dB, usually a small positive value in practice), and k is the sensitivity coefficient (unit: V/dB).

In the virtual driver, according to the noise_level value (unit: dB) in the environmental parameters, the output voltage is calculated based on Formula 15, and then converted into a digital quantity through the ADC. In order to ensure the authenticity of the simulation, appropriate Gaussian noise is superimposed on the final output, with a standard deviation of 1.5 dB, corresponding to the accuracy level of general consumer-grade noise sensors.

The virtual noise sensor also simulates the following characteristics:

(1) **Directivity**: Real microphones have a certain directivity, with different sensitivity to sounds from different directions. The virtual sensor introduces a simple directivity coefficient to simulate this physical characteristic.

(2) **Frequency Response**: The human ear has different sensitivity to sounds of different frequencies. A-weighting is the most commonly used weighting method for sound pressure levels. The virtual sensor outputs A-weighted sound pressure level by default, which is closer to the subjective feeling of the human ear.

(3) **Peak Hold**: Some noise sensors support peak detection functions, recording the maximum sound pressure level over a period of time. The virtual sensor implements an optional peak hold mode, recording and returning the maximum dB value within a specified time window.

4.7 Data Acquisition Loop Design

The data acquisition loop is the core business process of the entire virtual environmental monitoring system, responsible for driving each sensor to perform measurements at a fixed period and sending the collected data to the storage and display modules. A complete data acquisition cycle includes the following 6 steps:

1. **Read Environmental Parameters**: Read the current environmental parameter settings from the shared file env_control.txt, including temperature, humidity, air pressure, light, air quality, and noise level.

2. **Update Sensor Environment**: Pass the read environmental parameters to each virtual sensor instance, so that the sensors can generate simulated data according to the latest environmental settings.

3. **Execute Sensor Reading**: Call the read() method of each sensor in turn to obtain the current measured value. The reading order is usually: DHT11 (tempera-

ture/humidity) → BMP280 (pressure/temperature) → BH1750 (light) → MQ-135 (air quality) → Noise sensor.

4. **Data Formatting**: Convert the raw readings of each sensor into standard units (such as lux, hPa, ppm, dB, etc.), and organize them into dictionary or JSON format according to a predefined data structure.

5. **Data Storage**: Insert the formatted data record into the sensor_data table of the SQLite database, and update the latest data cache in memory at the same time.

6. **Alarm Check**: Traverse the alarm configuration of each sensor to check whether the current reading exceeds the threshold range. If exceeded, record the alarm log and update the alarm status.

In order to ensure the stability of the data acquisition cycle and avoid cycle jitter caused by excessive single acquisition time, a sampling period compensation mechanism is introduced. Let the target sampling period be T_{sample} (unit: seconds), and the actual elapsed time of a single acquisition be $t_{elapsed}$. Then the sleep time after this acquisition should be:

$$t_{sleep} = \max(0, T_{sample} - t_{elapsed}) \tag{16}$$

Formula 16 ensures that the actual period of the entire data acquisition loop will not be less than the target period. When the single acquisition time exceeds the target period, the loop will immediately enter the next round without additional waiting, to prioritize data real-time performance.

```
import time

SAMPLE_PERIOD = 2.0  # Sampling period 2 seconds

def data_collection_loop(sensors, db, alarm_config):
    """Data acquisition main loop"""
    while True:
        t_start = time.time()

        # Step 1: Read environmental parameters
        env = read_environment_control()

        # Step 2: Update sensor environment
        for sensor in sensors:
            sensor.set_environment(env)

        # Step 3: Execute sensor reading
```

```
        readings = {}
        for sensor in sensors:
            readings.update(sensor.read())

        # Step 4: Data formatting
        record = format_sensor_data(readings)

        # Step 5: Data storage
        db.insert_sensor_data(record)

        # Step 6: Alarm check
        check_alarms(record, alarm_config, db)

        # Period compensation
        t_elapsed = time.time() - t_start
        t_sleep = max(0, SAMPLE_PERIOD - t_elapsed)
        time.sleep(t_sleep)
```

Listing 6: Data acquisition loop core implementation

As shown in Code 6, the data_collection_loop function implements the complete 6-step data acquisition process. SAMPLE_PERIOD defines the target sampling period, and the cycle stability is ensured through the period compensation mechanism. Data is passed between steps through the readings dictionary, and the final formatted record is stored in the database and used for alarm checking.

5 Software System Design and Implementation

Based on the virtual STM32 hardware platform and sensor drivers, this chapter introduces the design and implementation of the upper-layer software system, including four core components: the data storage module, alarm system, Web visualization system, and control panel. These modules together constitute the complete intelligent environmental monitoring station application, realizing persistent data management, automatic detection of abnormal events, intuitive display of monitoring results, and remote control of environmental parameters.

5.1 Data Storage Module

Data storage is one of the core functions of any monitoring system, responsible for persistently saving collected sensor data, system operation status, and alarm event information

to support subsequent operations such as historical query, statistical analysis, and audit tracing. This system uses SQLite as the database engine, which has the advantages of zero configuration, single-file storage, and easy deployment.

5.1.1 Database Table Structure Design

The database contains three core tables: sensor_data (sensor data table), alarm_log (alarm record table), and system_log (system log table).

(1) **sensor_data table**: Used to store real-time collected data from each sensor, and is the table with the largest data volume. Its structure design is shown in Table 7, containing a total of 10 fields.

Table 7: sensor_data Table Structure

Field Name	Data Type	Constraint	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Auto-increment key
timestamp	DATETIME	NOT NULL DEFAULT CURRENT_TIMESTAMP	Acquisition times
temperature	REAL		Temperature valu
humidity	REAL		Humidity value, u
pressure	REAL		Pressure value, u
illuminance	REAL		Illuminance, unit
air_quality	REAL		Air quality index,
noise_level	REAL		Noise level, unit d
cpu_temp	REAL		MCU internal ter unit °C
data_source	TEXT	DEFAULT 'simulation'	Data source ident

The id field is an auto-increment primary key, ensuring that each record has a unique identifier; the timestamp field automatically records the data insertion time, facilitating query by time range; the six fields from temperature to noise_level correspond to the measured values of the six environmental parameters; the cpu_temp field records the internal temperature of the virtual MCU (simulated by the internal temperature sensor); the data_source field identifies the source of the data, with a default value of 'simulation', facilitating subsequent expansion to support real hardware data access.

(2) **alarm_log table**: Used to record the occurrence, duration, and recovery process of alarm events. Fields include: id (primary key), timestamp (alarm time), sensor_type (sensor type), alarm_type (alarm type: HIGH/LOW), threshold (threshold), actual_value (actual value), status (status: ACTIVE/RESOLVED), and resolved_at (recovery time).

(3) **system_log table**: Used to record key events and debugging information during system operation. Fields include: id (primary key), timestamp (log time), level (log level: DEBUG/INFO/WARNING/ERROR), module (module name), message (log content), and metadata (additional metadata, JSON format).

5.1.2 Python sqlite3 Interface

The sqlite3 module in the Python standard library provides a complete access interface to the SQLite database. Through this module, SQL statements can be executed, transactions managed, and query results obtained, without the need to install additional database drivers.

Database connections adopt the Context Manager pattern, ensuring that connections can be properly closed after each database operation to avoid resource leaks. At the same time, by setting the journal_mode to WAL (Write-Ahead Logging), concurrent read and write performance is improved, allowing the data acquisition thread and the Web service thread to access the database simultaneously without serious lock contention.

5.1.3 CRUD Operations

The data storage module encapsulates a complete CRUD (Create, Read, Update, Delete) operation interface:

(1) **Create**: The insert_sensor_data method receives a dictionary containing the readings of each sensor and inserts it into the sensor_data table. The insertion operation uses a Parameterized Query, effectively preventing SQL injection attacks.

(2) **Read**: Provides a variety of query methods, including: get_latest_reading() to obtain the latest record; get_readings_by_range(start, end) to obtain records within a specified time range; get_readings_by_sensor(sensor_type, limit) to obtain the most recent N records of a sensor.

(3) **Update**: The update_alarm_status method is used to update the status of alarm records (from ACTIVE to RESOLVED), and set the resolved_at field to the current time.

(4) **Delete**: The delete_old_readings(before_date) method is used to delete historical data before a specified date, supporting both scheduled cleanup and manual cleanup modes to prevent unlimited growth of the database file.

5.1.4 Statistical Queries

In addition to basic CRUD operations, the data storage module also provides rich statistical query functions, supporting aggregated analysis of historical data. Commonly used statistical queries include:

(1) **Average Query** (AVG): Calculate the average reading of a sensor within a specified time period, reflecting the overall level during that period.

(2) **Minimum Query** (MIN): Calculate the minimum reading of a sensor within a

specified time period, used to identify extreme low-value events.

(3) **Maximum Query** (MAX): Calculate the maximum reading of a sensor within a specified time period, used to identify extreme high-value events.

(4) **Standard Deviation Query** (STDEV): Calculate the standard deviation of a sensor's readings within a specified time period, reflecting the degree of data fluctuation.

The SQL template for statistical queries is as follows:

```
SELECT
    AVG(temperature) as avg_temp,
    MIN(temperature) as min_temp,
    MAX(temperature) as max_temp,
    AVG(humidity) as avg_humidity,
    MIN(pressure) as min_pressure,
    MAX(pressure) as max_pressure
FROM sensor_data
WHERE timestamp BETWEEN ? AND ?;
```

Listing 7: Statistical query SQL template

As shown in Code 7, statistical information of multiple sensors can be obtained at one time through SQL aggregate functions. The parameterized time range condition ensures the flexibility and security of the query.

5.2 Alarm System Design

The alarm system is an important component of the intelligent environmental monitoring station, responsible for real-time monitoring of the readings of each sensor, issuing alarm notifications in a timely manner when abnormal values are detected, and helping users quickly respond to environmental changes.

5.2.1 Alarm Threshold Configuration

Alarm thresholds are defined through the ALARM_CONFIG configuration dictionary, supporting independent high thresholds (high_threshold) and low thresholds (low_threshold) for each sensor. When a sensor reading exceeds the high threshold, a high-temperature/high-concentration alarm is triggered; when it falls below the low threshold, a low-temperature/low-concentration alarm is triggered.

The default alarm configuration example is as follows:

```
ALARM_CONFIG = {
    'temperature': {
        'high_threshold': 35.0,    # High temperature alarm threshold
            35C
```

```
        'low_threshold': 10.0,     # Low temperature alarm threshold 10C
        'unit': 'C',
        'name': 'Temperature'
    },
    'humidity': {
        'high_threshold': 80.0,    # High humidity alarm threshold 80%
        'low_threshold': 20.0,     # Low humidity alarm threshold 20%
        'unit': '% RH',
        'name': 'Humidity'
    },
    'pressure': {
        'high_threshold': 1050.0, # High pressure alarm threshold 1050 hPa
        'low_threshold': 980.0,    # Low pressure alarm threshold 980 hPa
        'unit': 'hPa',
        'name': 'Pressure'
    },
    'air_quality': {
        'high_threshold': 500.0,   # Air quality alarm threshold 500 ppm
        'low_threshold': None,     # No low threshold
        'unit': 'ppm',
        'name': 'Air Quality'
    },
    # ... other sensor configurations
}
```

Listing 8: Alarm threshold configuration example

As shown in Code 8, each sensor's configuration items include threshold values, units, names, etc. When a sensor's low_threshold is set to None, it means that low-value alarms are not monitored, and only high-value alarms are monitored.

5.2.2 Alarm Checking Logic

The check_all function is the core of the alarm system, responsible for traversing all configured sensors and checking whether the current readings exceed the threshold range. Its checking logic is as follows:

(1) **Traversal Check**: For each sensor configuration item in ALARM_CONFIG, obtain the current reading and the corresponding threshold setting.

(2) **High Threshold Judgment**: If the current reading is greater than high_threshold, a HIGH type alarm is triggered. If an ACTIVE status HIGH alarm already exists for this sensor, its actual value is updated; otherwise, a new alarm record is created.

(3) **Low Threshold Judgment**: If the current reading is less than low_threshold, a LOW type alarm is triggered. The judgment logic is similar to that of the high threshold.

(4) **Alarm Recovery**: If the current reading has returned to the normal range, and there was previously an ACTIVE status alarm, the alarm is marked as RESOLVED status, and the recovery time is recorded.

(5) **Persistence**: Newly created alarm records and status changes are persistently saved through the database interface to ensure that the alarm history is traceable.

```
def check_all(readings, config, db):
    """
    Check the alarm status of all sensors
    Parameters:
        readings: current sensor readings dictionary
        config:   alarm configuration dictionary
        db:       database operation instance
    """
    for sensor, cfg in config.items():
        value = readings.get(sensor)
        if value is None:
            continue

        # High threshold check
        high = cfg.get('high_threshold')
        if high is not None and value > high:
            trigger_alarm(db, sensor, 'HIGH', high, value)

        # Low threshold check
        low = cfg.get('low_threshold')
        if low is not None and value < low:
            trigger_alarm(db, sensor, 'LOW', low, value)

        # Recovery check
        if (high is None or value <= high) and \
           (low is None or value >= low):
            resolve_alarm(db, sensor)
```

Listing 9: Alarm checking core logic

As shown in Code 9, the check_all function implements a complete alarm checking

process. trigger_alarm and resolve_alarm are auxiliary functions, responsible for creating new alarms and resolving existing alarms, respectively.

5.2.3 Alarm Record Persistence

Each alarm record contains complete context information to facilitate post-analysis and auditing. The data model of the alarm record is shown in Table 8.

Table 8: alarm_log Table Structure

Field Name	Data Type	Constraint	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Auto-increment primary key
timestamp	DATETIME	NOT NULL	Alarm trigger time
sensor_type	TEXT	NOT NULL	Sensor type
alarm_type	TEXT	NOT NULL	Alarm type: HIGH/LOW
threshold	REAL	NOT NULL	Trigger threshold
actual_value	REAL	NOT NULL	Actual value at trigger time
status	TEXT	DEFAULT 'ACTIVE'	Status: ACTIVE/RESOLVED
resolved_at	DATETIME		Recovery time

5.3 Web Visualization System

The Web visualization system is the main interface for user interaction with the virtual environmental monitoring station, responsible for presenting the underlying collected sensor data in an intuitive and beautiful form, and providing functions such as environmental parameter control and alarm information viewing.

5.3.1 Flask Route Design

The back-end uses the Flask framework to provide Web services, defining multiple RESTful API endpoints for front-end calls. The functions of each endpoint are shown in Table 9.

Table 9: Flask RESTful API Route Design

Method	Path	Function	Description
GET	/	Home page	Returns the Web visualization page
GET	/api/sensors/latest	Latest data	Returns the latest readings of all sensors
GET	/api/sensors/history	Historical data	Returns historical data within a specified time range
GET	/api/alarms/active	Active alarms	Returns the list of currently unresolved alarms
GET	/api/alarms/history	Alarm history	Returns all alarm records (supports pagination)
POST	/api/control	Environment control	Receives environmental parameter setting commands
GET	/api/stats	Statistics	Returns statistical information for a specified period

5.3.2 RESTful JSON Response Format

All API endpoints adopt a unified JSON response format, containing three top-level fields: status, data, and message. The status field is 'ok' or 'error', indicating the request processing status; the data field contains the actual response data; the message field provides error description information when an error occurs.

Successful response example:

```
{
    "status": "ok",
    "data": {
        "temperature": 26.5,
        "humidity": 58.2,
        "pressure": 1013.8,
        "illuminance": 320.0,
        "air_quality": 120.5,
        "noise_level": 45.3,
        "timestamp": "2024-01-15T09:30:00"
    },
    "message": null
}
```

Listing 10: API successful response example

5.3.3 Chart.js Dual Line Chart Configuration

The front end uses the Chart.js library to draw real-time trend charts of sensor data. In order to display the comparison trends of two related sensors in the same chart, the system uses a dual line chart configuration. Taking the temperature and humidity trend chart as an example, its Chart.js configuration is as follows:

```
const ctx = document.getElementById('tempHumChart').getContext('2d');
const tempHumChart = new Chart(ctx, {
    type: 'line',
    data: {
        labels: [],       // Time label array
        datasets: [
            {
                label: 'Temperature (C)',
                data: [],
                borderColor: 'rgb(255, 99, 132)',
                backgroundColor: 'rgba(255, 99, 132, 0.1)',
                yAxisID: 'y-temp',
                tension: 0.3
            },
            {
                label: 'Humidity (% RH)',
                data: [],
                borderColor: 'rgb(54, 162, 235)',
                backgroundColor: 'rgba(54, 162, 235, 0.1)',
                yAxisID: 'y-humidity',
                tension: 0.3
            }
        ]
    },
    options: {
        responsive: true,
        interaction: { mode: 'index', intersect: false },
        scales: {
            'y-temp': {
                type: 'linear',
                display: true,
                position: 'left',
                title: { display: true, text: 'Temperature (C)' }
            },
            'y-humidity': {
                type: 'linear',
```

```
                display: true,
                position: 'right',
                title: { display: true, text: 'Humidity (% RH)' }
            }
        }
    }
});
```

Listing 11: Chart.js dual line chart configuration

As shown in Code 11, the dual line chart binds the two curves to the left and right Y axes respectively through the yAxisID property, avoiding data visualization distortion caused by different units of measurement. The tension property controls the smoothness of the curve. Setting it to 0.3 can maintain a clear trend while adding a certain visual smoothing effect.

5.3.4 1-Second Polling Mechanism

In order to achieve real-time data updates, the front end adopts a timed polling mechanism with a 1-second interval, periodically requesting the latest sensor data and alarm status from the back-end API. The implementation of the polling mechanism is based on JavaScript's setInterval function:

```
// Poll every 1000 milliseconds (1 second)
setInterval(() => {
    fetch('/api/sensors/latest')
        .then(r => r.json())
        .then(data => {
            if (data.status === 'ok') {
                updateSensorCards(data.data);
                updateCharts(data.data);
            }
        })
        .catch(err => console.error('Polling error:', err));

    fetch('/api/alarms/active')
        .then(r => r.json())
        .then(data => {
            if (data.status === 'ok') {
                updateAlarmPanel(data.data);
            }
        });
}, 1000);
```

Listing 12: Front-end polling mechanism implementation

As shown in Code 12, the front end simultaneously polls the latest sensor data and active alarm information to ensure that the data displayed on the interface is synchronized with the back-end status. Although the polling mechanism has slightly lower real-time performance compared to push technologies such as WebSocket, its implementation is simple and has good compatibility, which is sufficient for application scenarios with a 1-second update frequency. In the future, Server-Sent Events (SSE) or WebSocket can be considered to reduce network overhead and server load.

5.4 Control Panel Design

The control panel is the entry point for user interaction with the virtual environment, allowing users to adjust environmental parameters in real time through the Web interface and observe the response behavior of virtual sensors under different environmental conditions. This function is particularly important for teaching demonstrations and system debugging.

5.4.1 IPC Mechanism Design

Since the data acquisition loop of the virtual MCU runs in an independent Python thread, while the Flask Web service runs in the main thread, cross-thread communication is required between the two to pass control commands. This system adopts a file-sharing-based inter-process communication (IPC) mechanism, using the env_control.txt file as the data exchange medium.

The reasons for choosing file sharing as the IPC mechanism are as follows: First, it is simple to implement, without the need to introduce additional message queues or shared memory libraries; Second, the state is persistent. Even if the Web service restarts, the environmental parameter settings remain in the file; Third, it is easy to debug. Developers can directly view and edit the file content to test specific scenarios; Finally, it is cross-language compatible. If other language modules need to be introduced in the future, the file interface is the most universal communication method.

The env_control.txt file adopts a simple key-value pair format, with one parameter per line, in the format key=value:

```
temperature=25.0
humidity=60.0
pressure=1013.25
illuminance=500.0
air_quality=100.0
```

```
noise_level=40.0
wind_speed=5.0
```

Listing 13: Environmental control file format example

As shown in Code 13, the file contains 7 environmental parameters. The data acquisition loop reads this file before each sampling to update the environmental input of the virtual sensors; the Web back-end updates this file after receiving control commands to make the new settings take effect.

5.4.2 set Command Parsing

The control panel's API endpoint receives a POST request, and the request body contains the environmental parameters to be set. After parsing the request, the back-end writes the parameters into the env_control.txt file. The parsing logic of the set command supports partial updates, that is, users can update one or more parameters at the same time, and unspecified parameters remain unchanged.

```python
@app.route('/api/control', methods=['POST'])
def set_environment():
    """Set environmental parameters"""
    data = request.get_json()
    if not data:
        return jsonify({'status': 'error', 'message': 'No data'}),
            400

    # Read current environmental parameters
    env = read_env_file()

    # Update specified parameters
    valid_keys = ['temperature', 'humidity', 'pressure',
                  'illuminance', 'air_quality', 'noise_level',
                  'wind_speed']
    for key in valid_keys:
        if key in data:
            env[key] = float(data[key])

    # Write to file
    write_env_file(env)

    return jsonify({'status': 'ok', 'data': env})
```

Listing 14: Environmental parameter setting interface

As shown in Code 14, the set_environment function first reads the current environmental parameters, then traverses the valid parameter list, updates the parameter values specified in the request, and finally writes the updated parameters back to the file. Parameter type checking ensures that input values can be correctly converted to floating-point numbers.

5.4.3 Parameter Range Limits

In order to prevent users from entering unreasonable parameter values that cause abnormal sensor simulation, the system sets reasonable valid ranges for the 7 environmental parameters. When a submitted value exceeds the range, the back-end returns an error message, requesting the user to re-enter. The parameter range limits are shown in Table 10.

Table 10: Environmental Parameter Valid Ranges

Parameter Name	**Minimum**	**Maximum**	**Default**	**Unit**
temperature	−40.0	+85.0	25.0	°C
humidity	0.0	100.0	50.0	% RH
pressure	300.0	1100.0	1013.25	hPa
illuminance	0.0	65535.0	500.0	lux
air_quality	0.0	10000.0	100.0	ppm
noise_level	0.0	150.0	40.0	dB
wind_speed	0.0	100.0	5.0	m/s

The setting of parameter ranges refers to the measurement ranges of each sensor and physical rationality. For example, the temperature range corresponds to the operating temperature range of the BMP280; the humidity range is limited to 0–100% (physically impossible to exceed); the pressure range covers extreme situations from high mountains to sea level; the illuminance upper limit of 65535 lux corresponds to the maximum range of the BH1750 in H-Res mode. Through these limits, users can be effectively prevented from causing the system to output unreasonable data due to misoperation.

6 System Testing and Verification

System testing and verification is a key link to ensure software quality. Through comprehensive testing of each module such as the virtual STM32 hardware platform, sensor drivers, data storage, alarm system, and Web visualization, it is verified whether the system meets the design requirements and performance indicators. This chapter will introduce in detail the test environment, functional testing, performance testing, sensor

accuracy verification, and alarm function verification.

6.1 Test Environment

The test environment configuration of this system is shown in Table 11.

Table 11: System Test Environment Configuration

Item	**Configuration**
Operating System	Windows 11 Professional 64-bit
Processor	Intel Core i7-12700H @ 2.3 GHz
Memory	16 GB DDR5
Python Version	Python 3.11.6
Flask Version	Flask 3.0.0
SQLite Version	SQLite 3.42.0
Browser	Google Chrome 120.0

The tests were conducted on a laptop equipped with an Intel Core i7-12700H processor and 16 GB of memory, which represents the performance level of a current mainstream development environment. All tests were run locally, without involving network latency and remote server influence.

6.2 Functional Testing

Functional testing aims to verify whether the various functions of the system are correctly implemented according to the requirement specifications. Test cases cover the virtual MCU core, sensor drivers, Web access, alarm triggering, and control panel and other main functional modules. The functional test results are shown in Table 12.

Table 12: Functional Test Results

ID	Test Item	Test Steps	Expected Resu
FT-01	MCU Startup	Initialize VirtualSTM32 instance	System starts norm
FT-02	ADC Enable	Configure ADC clock and enable bit	ADC enters ready s
FT-03	DHT11 Read	Call dht11.read() method	Returns temperature and hu
FT-04	MQ-135 Read	Call read() after warm-up	Returns ppm concentrat
FT-05	BH1750 Read	Send measurement command and read	Returns lux illuminanc
FT-06	BMP280 Read	Read chip ID and get data	Returns pressure and ter
FT-07	Noise Read	Call noise.read() method	Returns dB valu
FT-08	Web Home Page Access	Browser accesses http://localhost:5000/	Page loads norma
FT-09	API Data Acquisition	GET request /api/sensors/latest	Returns JSON da
FT-10	High Temperature Alarm Trigger	Set temperature=40	Alarm record gener
FT-11	Low Pressure Alarm Trigger	Set pressure=920	Alarm record gener
FT-12	Control Panel Operation	Modify environmental parameters via Web interface	Parameter file upd

As shown in Table 12, all 12 functional tests were successfully passed, indicating that the various functions of the system have been correctly implemented according to the design requirements. Among them, FT-03 to FT-07 verified the basic reading functions of the five sensor drivers; FT-08 and FT-09 verified the availability of the Web service; FT-10 and FT-11 verified the threshold detection function of the alarm system; FT-12 verified the parameter setting function of the control panel.

6.3 Performance Testing

Performance testing aims to evaluate the response speed and resource consumption of the system in actual operation, ensuring that the system can meet the performance requirements of real-time monitoring.

6.3.1 API Response Time

Through Python's requests library, 100 consecutive requests were made to the Flask back-end API to measure the average response time of each endpoint. The test results are shown in Table 13.

Table 13: API Response Time Test Results

API Endpoint	Average Response Time	Minimum	Maximum
GET /api/sensors/latest	12.5 ms	8.2 ms	28.4 ms
GET /api/sensors/history	45.3 ms	32.1 ms	112.6 ms
GET /api/alarms/active	9.8 ms	6.5 ms	22.1 ms
POST /api/control	15.2 ms	10.3 ms	35.7 ms
GET /api/stats	38.6 ms	25.4 ms	98.3 ms

As shown in Table 13, the average response time of all API endpoints is less than 100 milliseconds, meeting the performance indicators set in the non-functional requirements. Among them, the /api/sensors/latest and /api/alarms/active endpoints have the shortest response times (about 10 ms), because they only involve single-record queries; the /api/sensors/history and /api/stats endpoints have slightly longer response times (about 40–45 ms), because they need to process time range queries and aggregate calculations, involving more database operations.

6.3.2 Acquisition Cycle Stability

The stability of the data acquisition cycle directly affects the smoothness of the trend chart and the accuracy of alarm judgment. By recording the actual elapsed time of 100 consecutive acquisition cycles, the stability and deviation of the cycles are analyzed. The test condition is a target period $T_{sample} = 2.0$ seconds.

The test results show that the average value of the acquisition cycle is 2.003 seconds, the standard deviation is 0.018 seconds, and the maximum deviation is +0.052 seconds. The cycle deviation mainly comes from fluctuations in the execution time of a single database write and alarm check. When there is a large amount of historical data in the database, the time consumption of the insert operation increases slightly, but due to the adoption of WAL mode, the impact of lock contention is small. Overall, the stability of the acquisition cycle meets the needs of real-time monitoring, and the trend chart can accurately reflect the changing trends of environmental parameters.

6.3.3 Memory Usage

The memory usage of the system is monitored through Python's tracemalloc and psutil libraries. After the system has been running continuously for 1 hour, the measured memory usage is shown in Table 14.

Table 14: Memory Usage Test Results

Module	**Memory Usage**
Flask Web Service	Approx. 45 MB
Virtual MCU and Sensor Instances	Approx. 12 MB
Data Acquisition Thread	Approx. 8 MB
SQLite Database Cache	Approx. 5 MB
Front-end Page Resources	Approx. 3 MB (Browser side)
Total	**Approx. 73 MB**

As shown in Table 14, the total memory usage of the system is approximately 73 MB, of which the Flask Web service occupies most of the memory (about 45 MB), which is mainly the memory overhead of the Python interpreter and the Flask framework itself. For modern computers, a memory usage of 73 MB is very small, and the system can run smoothly on resource-constrained devices (such as Raspberry Pi).

6.4 Sensor Precision Verification

Sensor precision is a key indicator to measure the credibility of the virtual environmental monitoring station. By comparing with the precision parameters published in real sensor datasheets, it is verified whether the simulation precision of the virtual sensors reaches a reasonable level.

6.4.1 Precision Comparison Analysis

The precision comparison between each virtual sensor and the corresponding real sensor is shown in Table 15.

Table 15: Virtual Sensor and Real Sensor Precision Comparison

Sensor	**Measurement Parameter**	**Real Precision**	**Simulated Precision**	**Deviatio**
DHT11	Temperature	±2°C	±0.8°C	Better than
DHT11	Humidity	±5% RH	±1.5% RH	Better than
MQ-135	Gas Concentration	±15%	±8%	Better than
BH1750	Illuminance	±20%	±5%	Better than
BMP280	Pressure	±1 hPa	±0.5 hPa	Better than
BMP280	Temperature	±0.5°C	±0.3°C	Better than
Noise Sensor	Sound Pressure Level	±3 dB	±1.5 dB	Better than

As shown in Table 15, the simulation precision of all virtual sensors is better than the nominal values in the corresponding real sensor datasheets. This is because virtual sensors

are not affected by factors such as temperature drift, aging, electromagnetic interference, and power supply fluctuations in the real world, and their output noise is only generated by the random number generator at the software level, so they have higher stability and consistency.

Although the simulated precision is higher than the real precision, this does not affect the teaching value and verification significance of the virtual platform. In teaching scenarios, higher simulated precision helps students clearly observe the relationship between sensor response and environmental parameters, without being disturbed by the noise and errors of real sensors. In actual application verification, the precision of virtual sensors can be made closer to the real level by increasing the noise standard deviation σ, thereby conducting tests closer to reality.

6.4.2 Long-term Stability Test

A continuous 24-hour stability test was performed on the virtual BMP280 barometric pressure sensor. The environmental parameters were set to constant values ($P = 1013.25$ hPa, $T = 25.0$°C), and the sampling period was 2 seconds. The test results show that a total of 43,200 data points were collected within 24 hours. The average value of the pressure readings is 1013.28 hPa, the standard deviation is 0.42 hPa, and the maximum deviation is +1.8 hPa/−1.5 hPa. The data distribution conforms to the characteristics of a normal distribution, and no obvious drift trend or systematic deviation was observed, verifying the stability of the virtual sensor during long-term operation.

6.5 Alarm Function Verification

The alarm function is an important component of the environmental monitoring station, directly related to whether users can detect environmental abnormalities in a timely manner and take countermeasures. This section verifies the correctness of the alarm system through two typical scenarios.

6.5.1 High Temperature Alarm Scenario

Test scenario: Set the ambient temperature to 40°C, which is higher than the high threshold of 35°C for the temperature alarm.

Test steps:

1. Set the temperature parameter to 40.0 through the control panel.
2. Wait for the data acquisition loop to execute at least one cycle (about 2 seconds).
3. Query the alarm_log table to check whether a new temperature alarm record has been generated.

4. Access the alarm panel of the Web interface to confirm that the alarm information is correctly displayed.

5. Restore the temperature to 25.0 and wait for the alarm to recover.

Test result: After setting temperature=40.0, a high temperature alarm was triggered within the second acquisition cycle. A new record was added to the alarm_log table with status 'ACTIVE' and alarm_type 'HIGH', with an actual_value of 40.1°C (including noise) and a threshold of 35.0°C. The alarm panel of the Web interface highlighted this alarm information in red. After restoring the temperature, the alarm status was automatically updated to 'RESOLVED' in about 2 seconds, and the resolved_at field recorded the recovery time.

6.5.2 Low Pressure Alarm Scenario

Test scenario: Set the ambient pressure to 920 hPa, which is lower than the low threshold of 980 hPa for the pressure alarm.

Test steps:

1. Set the pressure parameter to 920.0 through the control panel.

2. Wait for the data acquisition loop to execute at least one cycle.

3. Query the alarm_log table to check whether a new pressure alarm record has been generated.

4. Access the alarm panel of the Web interface to confirm that the alarm information is correctly displayed.

5. Restore the pressure to 1013.25 and wait for the alarm to recover.

Test result: After setting pressure=920.0, the system correctly triggered a low pressure alarm, with alarm_type 'LOW' and actual_value 919.6 hPa. The alarm panel displayed this alarm in orange (low alarms use orange, high alarms use red). After restoring the pressure, the alarm status was normally updated to 'RESOLVED'.

6.5.3 Multiple Concurrent Alarms Scenario

A scenario where multiple sensors trigger alarms simultaneously was further tested. The temperature was set to 42.0°C and the air_quality to 800 ppm, triggering a high temperature alarm and a high air quality alarm at the same time. The test results show that the system can correctly handle concurrent alarms. The two alarm records are stored independently in the database, and the alarm panel of the Web interface displays both alarm messages at the same time, with their respective recovery statuses updated independently. This test verifies the correctness and stability of the alarm system when handling complex scenarios.

6.6 Web Interface Display

The Web visualization interface is the main window for user interaction with the system. The rationality of its layout design and information display directly affects the user experience. The Web interface of this system mainly includes the following areas:

(1) **Sensor Data Cards**: The top of the page displays the current readings of each sensor in the form of 6 cards. Each card contains the sensor name, current value, unit, and trend arrow (rising/falling/flat state compared to the last reading). The card color is displayed as green (normal) or yellow (approaching threshold) according to whether the value is normal.

(2) **Temperature and Humidity Trend Chart**: The middle left of the page is a dual line chart for temperature and humidity. The red curve represents the temperature change trend, and the blue curve represents the humidity change trend. The horizontal axis is time, and the left and right vertical axes are the scales for temperature and humidity respectively. The chart supports hovering the mouse to view specific data points, and has smooth animation transition effects.

(3) **Air Quality and Illuminance Chart**: The middle right of the page is a dual line chart for air quality (purple curve) and illuminance (yellow curve), showing the real-time change trends of these two parameters. Since the numerical range of air quality is usually much smaller than that of illuminance, this chart also adopts a dual Y-axis design to avoid visual compression.

(4) **Statistics Information Panel**: The lower left of the page displays statistics for the past 1 hour, 24 hours, and 7 days, including the average, minimum, and maximum values of each sensor. Statistical data is dynamically obtained through the /api/stats interface, and users can choose different time ranges to view.

(5) **Alarm Panel**: The lower right of the page displays the current active alarm list and historical alarm records. Active alarms are highlighted in conspicuous colors (red/orange), including alarm type, sensor name, trigger time, and current value. Historical alarm records are arranged in reverse chronological order, supporting pagination browsing.

(6) **Control Panel**: The sidebar of the page provides sliders and input boxes for adjusting environmental parameters. Users can adjust 7 parameters in real time: temperature, humidity, pressure, illuminance, air quality, noise level, and wind speed. Each parameter displays the current value and valid range. Inputs exceeding the range will be rejected and a prompt will be given.

The Web interface adopts a responsive layout design, which can be well displayed on desktop browsers and mobile device screens. The overall color scheme uses light gray as the background, blue and green as the main colors, and red and orange for alarm states, with clear visual hierarchy and efficient information conveyance.

7 Conclusion and Future Work

7.1 Summary of Thesis Work

This paper conducts in-depth research and system implementation on the virtualization of the intelligent environmental monitoring station based on STM32, and mainly completes the following three aspects of work:

(1) **Constructed a high-fidelity virtual STM32 hardware platform**. Taking the STM32F103C8T6 microcontroller as the prototype, a complete set of virtual hardware platforms was designed and implemented, covering core peripheral modules such as Cortex-M3 core simulation, memory mapping system (Flash/SRAM/register area), SysTick timer, GPIO general-purpose input/output, ADC analog-to-digital converter, I2C bus controller, UART serial communication, and TIM general-purpose timer. The virtual platform is implemented in Python, with good code readability and cross-platform characteristics, providing a software environment in which embedded programs can run without real hardware. Experiments have verified that the behavior of the virtual peripherals is highly consistent with real STM32 register-level operations, and can meet the needs of embedded teaching and prototype verification.

(2) **Established a unified environmental sensor virtualization framework**. Based on the virtual hardware platform, virtual driver programs for five common environmental sensors, including the DHT11 temperature and humidity sensor, MQ-135 air quality sensor, BH1750 light sensor, BMP280 barometric pressure sensor, and noise sensor, were developed. A universal sensor modeling framework was proposed, including environmental parameter definitions, physical quantity conversion models, and the Gaussian noise model $Y = X + N(0, \sigma^2)$, ensuring that the simulated data conforms to physical laws and has a sense of reality. Each sensor driver fully simulates the communication protocol (single-wire, I2C), timing characteristics, and signal conditioning process, and the simulation accuracy reaches or is better than the nominal values in real sensor datasheets.

(3) **Implemented a complete Web data visualization and control system**. The back-end service was developed using the Python Flask framework, and RESTful API interfaces were designed for front-end calls; the SQLite database was used to store sensor data, alarm records, and system logs; the front end uses native HTML/CSS/JavaScript technology combined with the Chart.js chart library to realize real-time display of sensor data, historical trend analysis, intelligent alarm notifications, and remote control of environmental parameters. The system supports a file-sharing-based IPC mechanism, achieving closed-loop interaction between the Web control panel and the virtual MCU.

Experimental tests show that the virtual STM32 platform runs stably, the data acquisition cycle is accurate, the Web API response time is less than 100 milliseconds, and

the system memory usage is only about 73 MB. The alarm function can correctly respond to high and low threshold events, and the Web interface layout is reasonable and the information display is clear. This system provides a low-cost, high-flexibility experimental platform for embedded system teaching, and also provides an efficient software verification environment for IoT application prototype development.

7.2 Future Work Outlook

Although this system has implemented a relatively complete virtual environmental monitoring station, there is still room for further improvement and expansion in the following aspects:

(1) **FreeRTOS Real-Time Operating System Simulation**. The current virtual platform only simulates the bare-metal program running environment and does not yet support a real-time operating system (RTOS). In the future, the FreeRTOS task scheduler simulation can be introduced to realize RTOS core functions such as multi-task concurrent execution, semaphores, message queues, and memory management, enabling the virtual platform to run more complex embedded applications and be closer to industrial-level development scenarios.

(2) **MQTT Protocol Support**. The current system uses an HTTP polling mechanism to achieve front-end and back-end data synchronization. Although the implementation is simple, there are problems of insufficient real-time performance and additional network overhead. In the future, the MQTT (Message Queuing Telemetry Transport) protocol can be integrated, enabling the virtual MCU to push sensor data to an MQTT broker in a publish/subscribe mode, and the Web front end subscribes to related topics to receive real-time updates. The MQTT protocol is the de facto standard in the IoT field, and supporting this protocol will significantly enhance the industrial applicability of the system.

(3) **Expansion of More Sensor Types**. The current system supports five common environmental sensors. In the future, more types of sensor simulations can be expanded, including but not limited to: carbon dioxide sensors (such as MH-Z19C), PM2.5 particulate matter sensors (such as PMS5003), ultraviolet sensors (such as GUVA-S12SD), soil moisture sensors, and rain sensors. By enriching the types of sensors, the virtual environmental monitoring station can cover a wider range of application scenarios, such as agricultural greenhouse monitoring, smart city environmental monitoring, and industrial safety monitoring.

(4) **Low-Power Mode Simulation**. The STM32F103C8T6 supports multiple low-power modes (Sleep, Stop, and Standby). In actual battery-powered applications, low-power design is crucial. In the future, the entry and exit processes of various low-power modes can be simulated in the virtual platform, including clock shutdown, peripheral

power-off, and wake-up source configuration, helping learners understand and master embedded low-power design technologies.

(5) **Mobile App Development**. The current user interface of the system is a Web page. Although it has cross-platform advantages, the operation experience on mobile devices still has room for improvement. In the future, a supporting mobile app (based on the Flutter or React Native framework) can be developed, providing mobile features such as push notifications, voice broadcast, and geographic location association, making environmental monitoring more convenient and intelligent.

In summary, the research on the virtualization of the intelligent environmental monitoring station based on STM32 has important theoretical value and broad application prospects. With the continuous development of IoT technology and the growing demand for embedded talent cultivation, virtualization teaching platforms will play an increasingly important role in lowering the learning threshold, improving teaching efficiency, and promoting technological innovation.

References

References

[1] Liu Huoliang, Yang Sen. STM32 Library Development Practical Guide: Based on STM32F4[M]. Beijing: China Machine Press, 2019.

[2] Positive Atom. STM32F1 Development Guide: Library Function Version[M]. Guangzhou: Positive Atom, 2022.

[3] Wang Lihua, Zhang Minghui. Design and Implementation of Embedded System Virtual Experiment Platform[J]. Computer Education, 2021(5): 112–117.

[4] Chen Zhigang, Li Xiaofeng. Research Review of Intelligent Environmental Monitoring System Based on Internet of Things[J]. Transducer and Microsystem Technologies, 2020, 39(8): 1–4.

[5] Zhou Runjing, Zhang Lina. STM32 Simulation and Application Examples Based on Proteus[M]. Beijing: Publishing House of Electronics Industry, 2018.

[6] ARM Limited. Cortex-M3 Technical Reference Manual[EB/OL]. ARM Developer, 2021.

[7] STMicroelectronics. RM0008 Reference Manual: STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx advanced ARM-based 32-bit MCUs[EB/OL]. ST Community, 2021.

[8] Sensirion AG. SHT30-DIS Datasheet: Humidity and Temperature Sensor[EB/OL]. Sensirion, 2020.

[9] Bosch Sensortec. BMP280 Digital Pressure Sensor Datasheet[EB/OL]. Bosch, 2020.

[10] ROHM Semiconductor. BH1750FVI-E Datasheet: Digital 16bit Serial Output Type Ambient Light Sensor[EB/OL]. ROHM, 2019.

[11] Hanwei Electronics. MQ-135 Gas Sensor Datasheet[EB/OL]. Hanwei, 2018.

[12] Flask Documentation. Flask 3.0 Quick Start[EB/OL]. Pallets Projects, 2023. https://flask.palletsprojects.com/

[13] Mozilla Developer Network. Fetch API Web APIs[EB/OL]. MDN Web Docs, 2023. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

[14] Chart.js Documentation. Chart.js 4.x Getting Started[EB/OL]. Chart.js, 2023. https://www.chartjs.org/docs/

[15] Wang Zhihua, Liu Yang. Design and Implementation of IoT Environmental Monitoring System Based on MQTT Protocol[J]. Internet of Things Technologies, 2022, 12(3): 45–48.

[16] Li Jianguo, Zhang Wei. Research on the Application of Simulation Technology in Embedded System Teaching[J]. Journal of Electrical and Electronic Education, 2021, 43(2): 78–82.

[17] SQLite Consortium. SQLite Documentation[EB/OL]. SQLite, 2023. https://www.sqlite.org/docs.html

[18] Zhao Minghua, Sun Li. Application of Virtual Instrument Technology in Sensor Teaching[J]. Experimental Technology and Management, 2020, 37(6): 156–160.

[19] QEMU Project. QEMU Documentation[EB/OL]. QEMU, 2023. https://www.qemu.org/documentation/

[20] Yang Liqiang, Wang Haitao. Design of Remote Embedded Experiment Teaching Platform Based on Web[J]. Computer Engineering and Applications, 2019, 55(14): 245–250.